



KG COLLEGE OF ARTS AND SCIENCE

Autonomous Institution | Affiliated to Bharathiar University

Accredited with A++ Grade by NAAC

ISO 9001:2015 Certified Institution

KGISL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON
Job Application Tracking System for Students

Submitted by
SUDHARSAN S
25BCS156

Under the Guidance of
Mr.Aravind
Technical Trainer
Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled “**Job Application Tracking System for Students**” submitted in partial fulfillment of the requirements for the course **Java Programming** is a **record of my original work** carried out under the guidance of **Mr.Aravind**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

SUDHARSAN S

Coimbatore

CERTIFICATE

This is to certify that the project entitled “**Job Application Tracking System for Students**” is a **bona fide work** carried out by **Sudharsan (2025-B.Sc.Computer Science-’B’)** **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **Java Programming**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: COIMBATORE

Date: 18/04/2026

Submitted for the Viva-Voce Examination held on 18/04/2026

Guide:
Mr.Aravind
(Technical Trainer, KGCAS)

Head of the Department:
Dr.Boopalan
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	Introduction	6
1.1	Objective	6
1.2	Overview	6
1.3	Features	6
2	Program Analysis	7
2.1	Problem Definition	7
2.2	Project Overview	7
2.3	Scopes and Limitations	7
3	System Design	8
3.1	Modules	8
4	Methodology	9
4.1	Problem Analysis	9
4.2	System Design	9
4.3	Program Development	10
4.4	Testing and Validation	11
4.5	Result and Documentation	11
4.6	Algorithm	12
4.7	Flowchart	12
5	Source Code	14
6	Input and Output Screens	19
7	Discussion and Conclusion	20
7.1	Discussion	20
7.2	Conclusion	20
8	References	21

ABSTRACT

The Job Application Tracking System is a Java-based application designed to help students efficiently manage and track their job applications. Many students apply to multiple companies and often face difficulty in keeping track of application status, interview schedules, and outcomes. This system provides a centralized platform where users can store details such as company name, job role, and application status. The application supports functionalities like adding, viewing, updating, deleting, and searching job applications. The system is developed using Core Java concepts such as classes, objects, ArrayList, loops, and conditional statements. It is implemented as a menu-driven console application. The main objective of this system is to improve organization, reduce confusion, and help students manage their job search process effectively.

1. INTRODUCTION

1.1 Objective

- ☐ To develop a Java-based job tracking system
- ☐ To automate job application management
- ☐ To reduce manual errors
- ☐ To demonstrate Object-Oriented Programming concepts
- ☒ To provide an easy-to-use system for students

1.2 Overview

The system allows users to:

- Add job applications
- View all applications
- Update application status
- Delete applications
- Search by company name

All data is stored dynamically using ArrayList.

1.3 Features

- ☐ Add job applications
- ☐ View all records
- ☐ Update status (Applied, Interview, Selected, Rejected)
- ☐ Delete records
- ☐ Search by company
- ☐ Simple menu-driven interface

2. PROGRAM ANALYSIS

2.1 Problem Definition

Students face issues such as:

1. Forgetting applied companies
2. Losing track of status
3. No centralized record system

This system solves these problems by providing a structured tracking method.

2.2 Project Overview

The system is a console-based Java application that:

- Stores job data
- Allows CRUD operations
- Displays results instantly

2.3 Scopes and Limitations

Scope:

- Useful for students
- Can manage multiple job applications
- Can be extended to real systems

Limitations:

- No database (temporary storage)
- Console-based (no GUI)

3. SYSTEM DESIGN

3.1 Modules

1. Input Module

- Takes user input (ID, company, role, status)

2. Processing Module

- Performs operations:
 - Add
 - Update
 - Delete
 - Search

3. Output Module

- Displays job details

4. Storage Module

- Uses ArrayList

Module Interaction Overview

User Input



Input Module



Processing Module



Data Storage Module



Output Module



User

4. METHODOLOGY

The methodology describes the systematic approach followed to design, develop, and implement the Job Application Tracking System for Students. A structured development process was used to ensure that the system is efficient, reliable, and easy to use.

The development of this project includes the following stages:

- Problem Analysis
- System Design
- Program Development
- Testing and Validation
- Result and Documentation

4.1. Problem Analysis

In this phase, the problem was carefully studied to understand the need for a job tracking system.

Identified Problems:

- Students forget applied job details
- Difficulty in tracking application status
- No centralized system for managing applications
- Manual methods are inefficient

Solution:

A Java-based system was proposed to automate and simplify job application tracking.

4.2. System Design

In this stage, the overall structure of the system was designed before coding.

Design Features:

- Menu-driven interface
- Modular structure
- Use of ArrayList for storage

Modules:

- Input Module

- Processing Module
- Output Module
- Data Storage Module

The system flow was planned to ensure smooth interaction between modules.

4.3. Program Development

The system was implemented using Java programming language.

Development Steps:

- Created class JobApplication
- Implemented main class with menu options
- Used ArrayList for dynamic data storage
- Applied:
 - Loops (while, for)
 - Conditional statements (if, switch)
 - Methods for modular coding
- Key Operations:
 - Add application
 - View applications
 - Update status
 - Delete application
 - Search by company

4.4. Testing and Validation

Test Case	Expected Result
Add application	Data stored correctly
View applications	Displays all records
Update status	Status updated correctly
Delete application	Record removed
Search company	Correct result shown

Validation:

- Verified outputs for different inputs
- Ensured no runtime errors
- Checked user input handling

4.5. Result and Documentation

After successful testing, the results were analyzed and documented.

Results:

- System performs all operations correctly
- Data is displayed clearly
- User-friendly interface

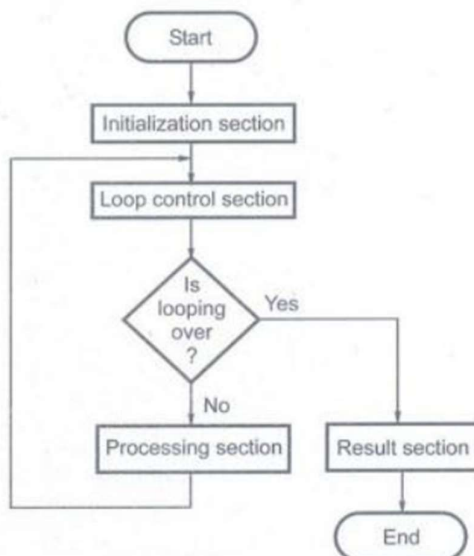
Documentation was prepared to explain:

- System design
- Code implementation
- Output results

4.6. Algorithm

1. Start
2. Display menu options
3. Accept user choice
4. Perform selected operation:
 - Add
 - View
 - Update
 - Delete
 - Search
5. Repeat until user exits
6. Stop

4.7. Flowchart



5. SOURCE CODE

```
import java.util.*;

public class Main {

    // Inner class for Job Application
    static class JobApplication {

        int id;

        String company;

        String role;

        String status;

        JobApplication(int id, String company, String role, String status) {

            this.id = id;

            this.company = company;

            this.role = role;

            this.status = status;

        }

        void display() {

            System.out.println("ID: " + id +

                               ", Company: " + company +

                               ", Role: " + role +

                               ", Status: " + status);

        }

    }

    static ArrayList<JobApplication> list = new ArrayList<>();

    static Scanner sc = new Scanner(System.in);
```

```
public static void main(String[] args) {  
    int choice;  
  
    do {  
        System.out.println("\n===== Job Application Tracker =====");  
        System.out.println("1. Add Application");  
        System.out.println("2. View Applications");  
        System.out.println("3. Update Status");  
        System.out.println("4. Delete Application");  
        System.out.println("5. Search by Company");  
        System.out.println("6. Exit");  
        System.out.print("Enter your choice: ");  
  
        choice = sc.nextInt();  
  
        switch (choice) {  
            case 1:  
                addApplication();  
                break;  
            case 2:  
                viewApplications();  
                break;  
            case 3:  
                updateStatus();  
                break;  
            case 4:  
                deleteApplication();  
                break;  
            case 5:  
                searchByCompany();  
        }  
    } while (choice != 6);  
}
```

```

        break;
    case 6:
        System.out.println("Thank you! Exiting...");
        break;
    default:
        System.out.println("Invalid choice!");
    }

} while (choice != 6);
}

static void addApplication() {
    System.out.print("Enter ID: ");
    int id = sc.nextInt();
    sc.nextLine(); // clear buffer

    System.out.print("Enter Company Name: ");
    String company = sc.nextLine();

    System.out.print("Enter Role: ");
    String role = sc.nextLine();

    System.out.print("Enter Status (Applied/Interview/Rejected/Selected): ");
    String status = sc.nextLine();

    list.add(new JobApplication(id, company, role, status));
    System.out.println("Application Added Successfully!");
}

static void viewApplications() {
    if (list.isEmpty()) {

```

```

        System.out.println("No applications found.");
        return;
    }

    System.out.println("\n--- All Applications ---");
    for (JobApplication app : list) {
        app.display();
    }
}

static void updateStatus() {
    System.out.print("Enter Application ID: ");
    int id = sc.nextInt();

    for (JobApplication app : list) {
        if (app.id == id) {
            sc.nextLine();

            System.out.print("Enter New Status: ");
            String newStatus = sc.nextLine();
            app.status = newStatus;

            System.out.println("Status Updated Successfully!");
            return;
        }
    }

    System.out.println("Application not found.");
}

static void deleteApplication() {
    System.out.print("Enter Application ID: ");
    int id = sc.nextInt();

```



```

Iterator<JobApplication> it = list.iterator();

while (it.hasNext()) {
    JobApplication app = it.next();
    if (app.id == id) {
        it.remove();
        System.out.println("Application Deleted Successfully!");
        return;
    }
}

System.out.println("Application not found.");
}

static void searchByCompany() {
    sc.nextLine();
    System.out.print("Enter Company Name: ");
    String company = sc.nextLine();

    boolean found = false;

    for (JobApplication app : list) {
        if (app.company.equalsIgnoreCase(company)) {
            app.display();
            found = true;
        }
    }

    if (!found) {
        System.out.println("No applications found for this company.");
    }
}
}

```

6. INPUT AND OUTPUT SCREENS

```
===== Job Application Tracker =====  
1. Add Application  
2. View Applications  
3. Update Status  
4. Delete Application  
5. Search by Company  
6. Exit  
Enter your choice: |
```

```
===== Job Application Tracker =====  
1. Add Application  
2. View Applications  
3. Update Status  
4. Delete Application  
5. Search by Company  
6. Exit  
Enter your choice: 1  
  
Enter ID: 101  
Enter Company Name: TCS  
Enter Role: Software Engineer  
Enter Status: Applied
```

```
Enter your choice: 2  
  
--- All Applications ---  
ID: 101, Company: TCS, Role: Software Engineer, Status: Applied
```

7. DISCUSSION & CONCLUSION

7.1 Discussion

The system successfully tracks job applications and reduces manual work. It demonstrates Java programming concepts effectively.

7.2 Conclusion

The project achieves its objective of managing job applications efficiently. It can be improved by adding GUI and database features.

8. REFERENCES

Books:

- Java – Herbert Schildt

Websites:

- Oracle Java Documentation
- GeeksforGeeks
- W3Schools
- Javatpoint